Assignment 4 solution

1. 【考点：文件操作与内存数据结构】进程执行 `open()` 系统调用打开一个已存在的文件时，操作系统会进行一系列内部处理。下列关于 `open()` 过程的描述中，正确的是（ ）

   A. `open()` 调用会将文件的全部实际数据内容直接一次性加载至物理内存缓冲区中
   B. `open()` 调用成功后，返回给用户进程的是一个整数（文件描述符），它对应于该进程打开文件表中的一个索引项
   C. 若多个独立进程同时打开同一个文件，则在内核的"系统打开文件表"中只会占用同一个表项，且共享同一个读写偏移量（File Offset）
   D. 文件当前的读/写指针（File Offset）保存在磁盘上的索引节点（Disk inode）中，以便关机后恢复

   【答案】：B
   【解析】：
   B 选项正确：`open()` 的核心作用是建立进程与文件的连接。成功后返回的文件描述符（File Descriptor）本质上是进程打开文件表（Process Open-File Table）的数组下标
   A 选项错误：`open()` 只会将文件的元数据（FCB/inode）载入内存，并不把文件的数据块载入内存（数据的读写由后续 `read()` / `write()` 触发）
   C、D 选项错误：不同进程打开同一个文件时，各自拥有独立的读/写偏移量（File Offset），以便独立读写。偏移量是内存动态状态，保存在内存打开文件数据结构中，绝不会保存在磁盘 inode 中

2. 【考点：文件的物理结构】某文件系统采用 UNIX 混合索引分配方式。磁盘块大小为 4KB，磁盘地址项占用 4B。每个文件的索引节点（inode）中包含 10 个直接地址项、1 个一次间接地址项、1 个二次间接地址项和 1 个三次间接地址项。则该文件系统支持的单文件最大逻辑长度约为（ ）

   A. 4MB
   B. 4GB
   C. 4TB
   D. 4PB

   【答案】：C
   【解析】：
   一个磁盘块可以存放的地址项数量为：4KB / 4B = 1024 = 2^{10} 个
   计算各级索引能支持的最大数据块数及容量：
   直接地址项：10 × 4KB = 40KB
   一次间接地址项：1024 × 4KB = 4MB
   二次间接地址项：1024 × 1024 × 4KB = 4GB
   三次间接地址项：1024 × 1024 × 1024 × 4KB = 2^{30} × 4KB = 4TB
   主导最大容量的是最高级（三次间接）索引，文件最大逻辑长度约为 4TB

3. 【考点：目录与链接】关于 UNIX/Linux 系统中的硬链接（Hard Link）与软链接/符号链接（Symbolic Link），下列说法正确的是（ ）

   A. 删除源文件后，指向该文件的硬链接依然有效，只要其 inode 中的引用计数（Link Count）不为 0，文件数据块就不会被释放
   B. 硬链接可以轻松跨越不同的文件系统（Mount 点）建立，而软链接只能在同一个物理分区内部建立
   C. 软链接文件的 inode 中直接记录了源文件的 inode 编号，因此访问软链接时无需进行路径检索
   D. 建立硬链接会在磁盘上创建一个全新的 inode 节点，并将其物理块指针指向源文件的数据块

【答案】：A

【解析】：

A 选项正确：硬链接与源文件共享同一个 inode，inode 内维护引用计数。删除源文件只是删除了一个目录项并将引用计数减 1。只要引用计数大于 0，实际数据和 inode 就会一直保留

B 选项错误：硬链接依靠 inode 编号寻址，而 inode 编号仅在单个文件系统内唯一，因此硬链接不能跨文件系统；软链接存储的是目标路径字符串，可以跨文件系统

C 选项错误：软链接有自己独立的 inode 和物理块，物理块内存储的是源文件的路径名字符串，访问时必须重新走一次完整的目录路径检索

D 选项错误：建立硬链接不会新建 inode，仅在目录中增加一条“文件名 → 原 inode 编号”的映射

4. 【考点：目录与文件元数据】UNIX 文件系统将传统的目录项分解为“文件名 + inode 编号”，并将文件的元数据（权限、大小、物理块指针等）集中保存在磁盘的磁盘索引节点（Disk inode）中。这种设计的主要目的是（ ）

   A. 彻底消除文件系统在分配外存时产生的外部碎片
   B. 极大地缩小目录项占用的空间，使单个磁盘块能装入更多目录项，从而大幅减少目录检索时的磁盘 I/O 次数
   C. 使得所有文件的数据块都能通过索引节点实现连续物理存储
   D. 简化系统调用的执行，免去访问文件系统超级块（Superblock）的开销

   【答案】：B

   【解析】：

   传统的文件控制块（FCB）体积较大（通常几百字节），一个磁盘块只能装几个 FCB。进行目录检索时，需要将大量包含冗余元数据的磁盘块读入内存。将 FCB 拆分为 inode 后，目录项仅需保留“文件名（如 14B）+ inode 号（如 2B）”，一个 4KB 的磁盘块可以容纳上百个目录项。这极大地减少了查找文件路径时需要读取的磁盘块数量（即减少了磁盘 I/O），提升了检索性能

5. 【考点：外存空闲空间管理】某磁盘文件系统采用位图（Bitmap）管理空闲磁盘块。已知磁盘块大小为 2KB，盘块号从 0 开始编号；位图的字长为 32 位（字号和位号均从 0 开始）。若系统当前分配了一个逻辑块号为 1050 的空闲磁盘块，则该块在位图中对应的字号（Word Number）和位号（Bit Number）分别是（ ）

   A. 字号 32，位号 26
   B. 字号 32，位号 25
   C. 字号 33，位号 26
   D. 字号 32，位号 27

   【答案】：A

   【解析】：

   盘块号 N = 1050，位图字长 W = 32（编号从 0 开始）
   字号 = ⌊ N / W ⌋ = ⌊ 1050 / 32 ⌋ = 32（即第 32 个字，整除结果为 32）
   位号 = N (mod W) = 1050 (mod 32) = 26（32 × 32 = 1024，1050 - 1024 = 26）
   对应字号 32，位号 26

6. 【考点：外存空闲空间管理】UNIX 文件系统采用成组链接法（Grouped Link）管理外存空闲块。假设每组最多记录 100 个空闲块块号。当系统申请分配一个空闲块时，若当前内存专用块（超级块中的空闲块栈）中只剩下最后一个空闲块号，此时系统的正确处理方式是（ ）

   A. 直接将该块分配给请求进程，同时宣布外存空间已耗尽
   B. 先将该块中存放的下一组空闲块信息读入内存专用块中，然后再将该块分配给请求进程
   C. 拒绝分配该块，将其永久锁定为系统保留块
   D. 立即触发磁盘碎片整理，重新将所有空闲块改用单链表法链接

   【答案】：B

   【解析】：

   成组链接法中，每组的第一个空闲块（栈底/栈顶）兼作“指针块”，其内部存储了下一组所有空闲块的块号和数量。当内存专用块中只剩这最后一个块号时，说明当前组可用块已用尽。系统必须先将该块内部记录的下一组空闲块信息复制加载到内存专用块中，然后再将这个物理块分配出去

7. 【考点：文件系统全局结构】关于文件系统在外存（磁盘）和内存中的全局结构布局（Layout），下列说法正确的是（ ）

   A. 主引导记录（MBR）可以存放在磁盘分区的任意空闲位置，由 BIOS 动态扫描查找
   B. 超级块（Superblock）包含文件系统的关键元数据（如块大小、空闲块数等），通常在文件系统挂载（Mount）时被读入内存
   C. 磁盘上的 inode 表（Inode Table）没有固定区域，散乱交织存放在各个文件的数据块内部
   D. 系统打开文件表（System-wide Open-File Table）保存在磁盘的分区引导块中，关机后不会丢失

   【答案】：B
   【解析】：
   B 选项正确：超级块存储了文件系统的全局控制元数据，在文件系统挂载时会被优先加载至内存，供内核实时维护
   A 选项错误：主引导记录（MBR）固定位于磁盘的第一个扇区（0号扇区）
   C 选项错误：磁盘分区有严格的布局划分，inode 表存放在固定的 inode 区内
   D 选项错误：系统打开文件表是纯粹的内存数据结构，断电后即消失

8. 【考点：虚拟文件系统】现代操作系统引入了虚拟文件系统（VFS，Virtual File System）以支持多种不同的底层文件系统。下列关于 VFS 的说法中，错误的是（ ）

   A. VFS 为上层应用提供了统一的文件系统接口（如 `read()`, `write()` 等系统调用）
   B. VFS 在内存中建立了抽象的数据结构模型，包括超级块对象（Superblock）、索引节点对象（Inode）、目录项对象（Dentry）和文件对象（File）
   C. 若底层硬件或文件系统格式（如 FAT32）本身不支持 UNIX 权限，VFS 能通过软件机制强行在外存磁盘上修改并写入 UNIX 风格的物理权限位
   D. 当进程打开文件时，VFS 会通过其维护的函数指针表（操作集）将通用调用映射到具体文件系统的底层处理函数

   【答案】：C
   【解析】：
   C 选项说法错误：VFS 是位于内存中的抽象与转换层，它无法改变外存（磁盘）上已有的物理数据结构或文件系统的固有格式限制。若底层文件系统格式（如 FAT32）在磁盘设计上就不支持某种特性（如 UNIX 权限），VFS 最多在内存中进行模拟映射，而无法直接在磁盘的 FAT32 物理结构中“凭空创造”新的权限字段

9. 【考点：文件系统挂载】操作系统通过挂载（Mounting）将新文件系统整合到现有的树形目录结构中。下列关于挂载操作的描述中，正确的是（ ）

   A. 挂载操作会将目标分区中的所有文件数据物理复制到主文件系统的根目录下
   B. 挂载点（Mount Point）通常是一个已存在的空目录；挂载成功后，访问该目录实际访问的是新挂载文件系统的根目录
   C. 挂载状态信息永久固化在磁盘的超级块中，无需操作系统内存参与维护
   D. 挂载允许不同的 CPU 架构在不通过操作系统内核的情况下直接读写同一块物理磁盘

   【答案】：B
   【解析】：
   挂载是将一个存储设备上的文件系统附加到主树形目录结构某个目录（挂载点）的过程。挂载成功后，挂载点原本的内容会被隐藏，用户通过该挂载点目录访问的将是被挂载文件系统的根节点（指向其根目录的 inode/dentry）。这是一种逻辑映射，不发生任何实际物理数据的复制

10. 【考点：文件的逻辑结构与保护机制】关于文件的逻辑结构与文件保护机制，下列说法正确的是（ ）

    A. 访问控制列表（ACL）是针对每个用户建立的，列表中详细记录了该用户能够访问的所有文件及其权限
    B. UNIX 系统的三组权限位（所有者 Owner / 组 Group / 其他人 Others）是一种简化的访问控制列表，其权限信息保存在文件的 inode 中

C. 口令保护机制是将用户密码明文写在文件数据块的头部，由用户程序读取并自行比对校验
D. 文件的读/写权限控制属于文件的物理保护，目的在于防止高强度磁场干扰损坏磁盘数据

【答案】：B
【解析】：
B 选项正确：UNIX 的 `rwx` 权限控制（如 `chmod` 管理的 9 个比特位）是访问控制列表（ACL）的一种高效紧凑实现，直接保存在文件的 FCB / inode 中
A 选项错误：访问控制列表（ACL）是针对每个文件/目录建立的（记录哪些用户对该文件有什么权限）；而针对每个用户建立的权限清单称为“能力表（Capability List）”
C 选项错误：口令保护由操作系统内核统一校验，密码绝不会明文裸露在用户文件数据块中
D 选项错误：读/写权限控制属于逻辑保护；物理保护是指防止硬件物理损坏（如防潮、防磁、冗余备份等）

11. 【考点：树形目录】在 Linux 服务器运维或构建容器镜像（如 Docker Image）时，为了节省磁盘空间并实现多版本共享，工程上广泛使用了文件链接（Link）技术。关于树形目录结构下的硬链接（Hard Link）与软链接（Symbolic Link / Symlink），下列说法完全正确的是（ ）

    A. 创建一个软链接时，操作系统不会为该软链接分配新的 inode（索引节点），而是直接在目录项中记录原文件的路径字符串
    B. 硬链接的本质是多个不同的目录项指向同一个底层 inode。因此，硬链接不允许跨越不同的物理文件系统（Partitions）创建，以防止 inode 编号冲突
    C. 当用户通过命令 `rm` 删除源文件时，指向该文件的软链接和硬链接都会立刻失效，尝试访问它们将引发“文件不存在”的错误
    D. 为了防止在遍历树形目录树时陷入无限死循环，现代操作系统不仅禁止为目录创建软链接，同时也严禁为普通文件创建硬链接

    【答案】： B
    【解析】：
    B 选项完全正确：硬链接的核心原理是“多目录项映射到同一 inode”。因为不同的文件系统（磁盘分区）各自维护一套独立的 inode 编号空间，跨文件系统建立硬链接会导致 inode 编号的解析混乱（例如分区 A 的 inode 100 和分区 B 的 inode 100 是两个不同的文件）。因此，操作系统严格限制硬链接只能在同一个文件系统内部创建
    A 选项错误：创建一个软链接（符号链接）时，操作系统必须为其分配一个新的、独立的 inode 以及对应的数据块。只是这个新文件的数据块中存储的内容是“目标文件的绝对或相对路径字符串”
    C 选项错误：删除源文件仅仅是删除了源文件所在目录的一个目录项，并将该 inode 的引用计数（Link Count）减 1。对于硬链接，只要引用计数不为 0，底层数据和 inode 依然完好无损，硬链接仍可正常访问；对于软链接，由于其依赖路径字符串寻找原文件，原路径失效后软链接才会变成“死链接（Dangling Link）”
    D 选项错误：操作系统禁止为目录创建硬链接（为了防止文件系统目录树中出现环路），但允许为目录创建软链接。当然，为普通文件创建硬链接是完全合法的常规操作

12. 【考点：文件的操作（建立，删除，打开，关闭，读，写）】在编写处理海量网络连接与日志写入的高并发服务端程序时，工程师常会遇到 `Too many open files` 的系统报错。理解操作系统内核在内存中维护的文件表结构，是解决此类资源泄漏问题的关键。关于文件操作（尤其是“打开”与“关闭”）及其内存数据结构，下列描述完全正确的是（ ）

    A. 当应用程序调用 `open()` 系统调用时，操作系统不仅会解析文件路径定位到磁盘上的 inode，还会立刻将该文件的所有业务数据从磁盘块读取到用户态内存中，以便后续的 `read()` 操作全速执行
    B. 如果两个独立运行的进程各自调用 `open()` 打开了磁盘上的同一个文件，它们在操作系统的“系统级打开文件表（System-wide Open File Table）”中必定共享同一个表项，以此保证文件读写指针的同步
    C. 操作系统在内存中维护了“进程级打开文件表”和“系统级打开文件表”。调用 `close()` 关闭文件描述符仅仅是删除进程级表项并使系统级表项的引用计数减 1，只有当引用计数降为 0 时，才会真正清理系统级表项

D. 文件的元数据（如文件大小、创建时间、读写权限）只有在程序实际发起 `read()` 或 `write()` 请求时，才会被操作系统从磁盘加载到内存中的“活动 inode 表”里

【答案】： C

【解析】：

C 选项完全正确：文件描述符（FD）的底层管理涉及三级结构：进程打开文件表 → 系统打开文件表 → 内存 inode 表。多个进程可以独立打开同一文件，每次调用 `close()` 只会切断本进程的 FD 映射，并将系统表项的引用计数（或最终 inode 的引用计数）减 1。只有当计数归零，内核才会真正释放相关的内存管理结构。这是防止其他仍在操作该文件的进程崩溃的关键保护机制

A 选项错误：`open()` 操作仅仅是操作元数据。它的主要工作是查找目录，将磁盘上的 inode 复制到内存中，并建立相关的表项及文件描述符，绝对不会主动将文件的实际数据块内容读入内存（数据通常在第一次 `read()` 时触发缺页或通过预读机制加载）

B 选项错误：如果两个进程各自独立调用 `open()` 打开同一文件，内核会在“系统级打开文件表”中为它们创建两个完全独立的表项。这意味着它们拥有各自独立的“当前文件读写指针（File Offset）”，并不会自动同步（除非通过 `fork` 继承或显式传递 FD）。但这两个独立的系统表项最终会指向内存中的同一个 vnode/inode 表项

D 选项错误：文件的元数据（inode 信息）在执行 `open()` 操作时就已经被完整地从外存加载到了内存的活动 inode 表（Active Inode Table）中，供后续的权限检查和读写寻址使用，而不是等到读写时才加载

13. 【考点：文件的物理结构】无论是多媒体文件系统还是分布式数据库的本地存储引擎，文件物理结构的选择直接决定了系统的 I/O 吞吐量和随机访问延迟。关于文件的连续分配、链接分配以及基于 UNIX 的混合索引分配，下列说法完全正确的是（ ）

A. 连续分配（Contiguous Allocation）能够保证文件在磁盘上的物理块是紧密相连的，因此它不仅支持最高效的随机访问，而且在文件需要动态扩展体积时也极为方便，极少产生外部碎片

B. 采用隐式链接分配（Implicit Linked Allocation）结构的文件，其所有的目录项中只需记录文件的起始块号和结束块号。若要访问逻辑块号为 N 的数据（无内存缓存），必须严格产生 N 次磁盘 I/O

C. 在 UNIX 混合索引分配机制中，一个 inode 通常包含直接指针、一级间接指针、二级间接指针等。由于引入了多级间接块，访问大型文件末尾数据的速度一定会比访问小型文件（仅用直接指针）更快，因为多级缓存命中率更高

D. 索引分配解决了连续分配的碎片问题和链接分配不能随机访问的问题，但其致命代价是即使一个文件只有几个字节大小，操作系统也必须为其在磁盘上额外分配一整个物理索引块来存放其实根本用不到的索引指针

【答案】： B

【解析】：

B 选项完全正确：隐式链接分配中，每个物理磁盘块的末尾隐藏了一个指向下一块的指针。它完全不支持随机访问。要访问逻辑上的第 N 块（从 1 开始算），操作系统必须先读入第 1 块获取第 2 块的地址，再读入第 2 块……依此类推。因此，必须从头遍历，产生 N 次磁盘 I/O（假设没有提前缓存到内存）

A 选项错误：连续分配确实支持最高效的顺序和随机访问，但它的致命缺点是极难进行文件的动态扩展（如果文件末尾没有连续的空闲块，就必须搬迁整个文件），并且经过频繁的文件删改后，磁盘上会产生大量的外部碎片

C 选项错误：UNIX 混合索引访问大型文件末尾数据时，需要依次读取一级索引块、二级索引块等，这意味着逻辑-物理地址转换过程本身就需要额外的磁盘 I/O 访存。因此，访问大型文件深层数据的延迟一定大于访问仅需直接指针的小型文件

D 选项错误：在 UNIX 混合索引分配中，如果文件极小（例如几十 KB），其所有物理块号可以直接存放在 inode 自带的 10~12 个直接指针（Direct Pointers）中。此时，根本不需要在磁盘上额外分配任何一级或多级间接索引块。这正是混合索引兼顾小文件效率与大文件容量的精妙之处

14. 【考点：文件系统的全局结构】在 Linux 及容器生态中，用户可以毫无感知地将本地的 Ext4、网络文件系统（NFS）、甚至内存文件系统（tmpfs）挂载（Mount）到同一个目录树下。实现这一特性的核心是虚拟文件系统（VFS）。下列关于文件系统全局结构、VFS 及挂载机制的描述，完全正确的是（ ）

A. VFS 向上的用户空间提供了统一的 POSIX API（如 `open`、`read`），这意味着所有接入 VFS 的底层物理文件系统（如 FAT32、Ext4）在磁盘扇区上的物理数据布局必须完全一致
B. 虚拟节点（vnode）是 VFS 为了屏蔽不同底层文件系统差异而在内存中专门构建的抽象数据结构，它只存在于内存中，一旦系统断电，vnode 就会随之消失，而不会持久化到磁盘上
C. 当执行文件系统挂载（Mounting）操作时，如果挂载点（Mount Point）目录中原本存在其他文件，那么挂载成功后，这些原文件将被底层磁盘控制器物理擦除，以释放空间给新的文件系统
D. 文件系统的"超级块（Superblock）"通常存放在内存中，用于记录当前系统中所有已经打开的文件描述符及其读写偏移量，是操作系统进行进程同步的核心

【答案】： B
【解析】：
B 选项完全正确：VFS 的核心思想是面向对象的设计。`vnode`（在 Linux 中常统一称为 VFS inode）是 VFS 抽象出来代表一个文件的内存结构。它封装了对文件操作的函数指针（如 `vnode->read()`）。`vnode` 是纯粹的内存中结构，由内核在打开文件时动态生成；而具体文件系统的 `inode`（如 Ext4 inode）才是持久化存储在磁盘上的真实元数据
A 选项错误：VFS 的伟大之处恰恰在于它允许接入物理布局完全不同的底层文件系统。VFS 向上提供统一接口，向下要求各底层文件系统提供驱动函数来适配 VFS 的调用。底层物理结构（ Ext4 的多级索引 vs FAT32 的链接表）可以天差地别
C 选项错误：挂载操作会将一个新的文件系统连接到现有的目录树（挂载点）上。如果挂载点原先有内容，这些内容不会被擦除，而是被内核暂时隐藏（覆盖可见性）。当该文件系统被卸载（Unmount）后，原挂载点目录下的内容会立刻重新显现
D 选项错误：超级块（Superblock）是存储在磁盘特定位置（并在挂载时读入内存）的全局元数据，它记录了该物理文件系统本身的信息（如总块数、空闲块数、块大小、魔数等）。记录"已打开文件及其偏移量"的是内存中的打开文件表，与磁盘级的超级块毫不相干

15. 【考点：外存空闲空间管理办法】在开发大规模分布式文件系统（如 HDFS）或高性能本地 SSD 存储层时，高效地管理动辄 PB 级别的空闲磁盘块是核心挑战。针对不同的外存空闲空间管理办法（如空闲表、空闲链表、位示图），下列工程判断完全正确的是（ ）

A. 空闲表法（Free-Space Table）本质上就是为所有连续的空闲区建立一张索引表，这种方法与内存管理中的动态分区分配思想高度相似，非常适合于系统需要频繁分配大段物理上连续的磁盘空间的场景
B. 在位示图（Bitmap）法中，如果位示图的一个二进制位值为 '0' 代表盘块空闲，那么当系统频繁分配大量文件时，操作系统只需利用 CPU 强大的浮点运算单元即可快速定位空闲块，这使得其计算复杂度远低于其他方法
C. 成组链接法（Grouped Linked List，常用于 UNIX）由于将所有的空闲块地址集中存储在文件系统的外存超级块中，因此每次分配物理块时都不会产生任何额外的磁盘 I/O 开销
D. 空闲链表法（包括空闲盘块链和空闲盘区链）的最大优势在于其数据结构可以完全在内存中动态生成和维护，系统在格式化磁盘时根本不需要在物理磁盘扇区中预留任何空间来存放这些链表指针

【答案】： A
【解析】：
A 选项完全正确：空闲表法记录了每一个连续空闲区的起始盘块号和盘块数量。这与内存管理中用空闲分区表管理可变分区的机制如出一辙。当系统需要申请连续 N 个物理块时，只需在空闲表中采用 First Fit 或 Best Fit 算法扫描即可，非常适合分配大段连续存储空间（例如存放完整的媒体文件）
B 选项错误：位示图通过二进制位（0或1）映射物理块的空闲状态。在底层硬件实现中，寻找连续的 '0' 依赖的是 CPU 的位运算（Bitwise Operations，如位掩码、移位）以及特定的寻找第一个零位的硬件指令。CPU 的浮点运算单元（FPU）专门用于处理带有小数点的 IEEE-754 浮点数，与

离散的布尔位运算毫无关系

C 选项错误：成组链接法的核心是将空闲盘块按组（例如 100 个一组）进行组织，第一组的盘块号存放在内存的超级块中。当第一组分配耗尽时，系统必须去读取磁盘上存放着下一组指针信息的那个物理块，将其调入内存来覆盖当前的空闲盘块号栈。因此，跨组分配时必然会发生磁盘 I/O

D 选项错误：空闲链表法的指针（如指向下一个空闲块的物理块号）是直接存储在各个空闲的物理磁盘块自身内部的（或者占用磁盘上的专用区域构建盘区链表）。断电重启后，这些链表结构必须能从磁盘中恢复，因此绝对不可能“完全在内存中动态生成且不占用物理磁盘空间”

16. **68．A**

当用户首次对某文件发出操作请求时，需要先用系统调用 open()将该文件打开。所谓“打开”，是指系统利用文件名检索到指定文件的目录项后，将文件的属性（包括该文件在外存中的物理位置）从外存复制到内存的打开文件表的一个表目中，并给用户返回一个文件描述符 fd。此后（只要文件未被关闭），用户对该文件的所有操作就都是通过文件描述符 fd 来进行的。

17. **69．C**

文件分配表（FAT）通过一个与逻辑块一一对应的表项数组，既描述文件数据块的链接关系，又标识每个块的使用状态（如空闲、已分配或坏块），进而直接支持空闲空间的跟踪与分配。相比之下，目录仅用于建立文件名到元数据（如 FCB 或 inode）的映射；文件控制块（FCB）或 inode 存储单个文件的属性及物理位置信息；系统打开文件表则维护当前被进程打开的文件状态。这三者均不负责记录外存空闲空间的使用情况。

18. **70．C**

在采用目录与索引节点（inode）分离管理的文件系统中，新建文件 F 时，文件系统首先分配一个空闲 inode，并初始化其元数据字段，包括文件类型、访问权限、时间戳以及数据块指针等；随后，在所属目录文件中新增一个目录项，该目录项仅包含文件名 F 及其对应的 inode 号。由于访问权限等属性信息始终保存在 inode 中，目录项并不包含这些内容，因此目录文件不会记录文件的访问权限信息。综上，文件系统在创建文件时不会在目录文件中写入 F 的访问权限信息。

19. **17．A**

位图法利用一个二进制位来表示磁盘中一个盘块的使用情况，磁盘上的所有盘块都有一个二进制位与之对应。当其值为“0”时，表示对应的盘块空闲；当其值为“1”时，表示已分配。位图所占空间的大小只取决于外存空间的总大小（盘块数量），而与当前空闲块数量无关。

20. **18．C**

虚拟文件系统（VFS）是操作系统内核中的一个抽象层，其核心目标是向上层应用程序提供统一的文件操作接口（如 open、read 等），进而屏蔽底层各类文件系统（如 ext4、NTFS 等）在实现细节上的差异。VFS 并不是一种在虚拟内存中实现的文件系统，也不存储实际数据，仅作为软件抽象存在。它不改变底层文件系统的数据组织方式，因此无法提升文件访问速度，读/写性能仍由具体文件系统结构及存储设备决定。此外，VFS 的设计支持多种文件系统类型，包括本地文件系统和网络文件系统（如 NFS），能够透明地访问远程文件。因此，只有关于 VFS 提供统一接口的描述准确反映了其本质功能。

21. **10．【解答】**

1）在磁盘中连续存放（采取连续结构），磁盘寻道时间更短，文件随机访问效率更高；在 FCB 中加入的字段为<起始块号，块数>或<起始块号，结束块号>。

2）将所有的 FCB 集中存放，文件数据集中存放。这样在随机查找文件名时，只需访问 FCB 对应的块，可减少磁头移动和磁盘 I/O 访问次数。

22.

11. 【解答】

1）文件系统中所能容纳的磁盘块总数为 4TB/1KB = 2^{32}。要完全表示所有磁盘块，索引项中的块号最少要占 32/8 = 4B。而索引表区仅采用直接索引结构，因此 512B 的索引表区能容纳 512B/4B = 128 个索引项。每个索引项对应一个磁盘块，所以该系统可支持的单个文件最大长度是 128×1KB = 128KB。

2）这里考查的分配方式不同于我们熟悉的三种经典分配方式，但题目中给出了详细的解释。所求的单个文件最大长度一共包含两部分：预分配的连续空间和直接索引区。

连续区块数占 2B，共可表示 2^{16} 个磁盘块，即 2^{26}B。直接索引区共 504B/6B = 84 个索引项。所以该系统可支持的单个文件最大长度是 2^{26}B + 84KB。

为了使单个文件的长度达到最大，应使连续区的块数字段表示的空间大小尽可能接近系统最大容量 4TB。分别设起始块号和块数占 4B，这样起始块号可以寻址的范围是 2^{32} 个磁盘块，共 4TB，即整个系统空间。同样，块数字段可以表示最多 2^{32} 个磁盘块，共 4TB。

23. **12.** 【解答】

1）系统采用顺序分配方式时，插入记录需要移动其他的记录块，整个文件共有 200 条记录，要插入新记录作为第 30 条，而存储区前后均有足够的磁盘空间，且要求最少的访问存储块数，则要把文件前 29 条记录前移，若算访盘次数，则移动一条记录读出和存回磁盘各是一次访盘，29 条记录共访盘 58 次，存回第 30 条记录访盘 1 次，共访盘 59 次。

F 的文件控制区的起始块号和文件长度的内容会因此改变。

2）文件系统采用链接分配方式时，插入记录并不用移动其他记录，只需找到相应的记录，修改指针即可。插入的记录为其第 30 条记录，因此需要找到文件系统的第 29 块，一共需要访盘 29 次，然后把第 29 块的下块地址部分赋给新块，把新块存回磁盘会访盘 1 次，然后修改内存中第 29 块的下块地址字段，再存回磁盘，一共访盘 31 次。

4B 共 32 位，可以寻址 2^{32} = 4G 块存储块，每块的大小为 1KB，即 1024B，其中下块地址部分占 4B，数据部分占 1020B，因此该系统的文件最大长度是 4G×1020B = 4080GB。

24. **13.** 【解答】

1）两个目录文件 dir 和 dir1 的内容如下表所示。

dir目录文件

文件名	簇号
dir1	48

dir1目录文件

文件名	簇号
file1	100
file2	200

2）FAT 的簇号为 2 个字节，即 16 比特，因此在 FAT 表中最多允许 2^{16}（65536）个表项，一个 FAT 文件最多包含 2^{16}（65536）个簇。FAT 的最大长度为 2^{16}×2B = 128KB。文件的最大长度是 2^{16}×4KB = 256MB。

3）在 FAT 的每个表项中存放下一个簇号。file1 的簇号 106 存放在 FAT 的 100 号表项中，簇号 108 存放在 FAT 的 106 号表项中。

4）先在 dir 目录文件里找到 dir1 的簇号，然后读取 48 号簇，得到 dir1 目录文件，接着找到 file1 的第一个簇号，据此在 FAT 里查找 file1 的第 5000 个字节所在的簇号，最后访问磁盘中的该簇。因此，需要访问目录文件 dir1 所在的 48 号簇，及文件 file1 的 106 号簇。

25. **14.** 【解答】

1）簇大小为 4KB，每个地址项长度为 4B，因此每簇有 4KB/4B = 1024 个地址项。最大文件的物理块数可达 8 + 1×1024 + 1×1024^2 + 1×1024^3，每个物理块（簇）大小为 4KB，因此最大文件长度为(8 + 1×1024 + 1×1024^2 + 1×1024^3)×4KB = 32KB + 4MB + 4GB + 4TB。

2）文件索引节点总个数为 1M×4KB/64B = 64M，5600B 的文件占 2 个簇，512M 个簇可存放的文件总个数为 512M/2 = 256M。可表示的文件总个数受限于文件索引节点总个数，因此能存储 64M 个大小为 5600B 的图像文件

3）文件 F1 的大小为 6KB < 4KB×8 = 32KB，因此获取文件 F1 的最后一个簇的簇号只需要访问索引节点的直接地址项。文件 F2 大小为 40KB，4KB×8 < 40KB < 4KB×8 + 4KB×1024，因此获取 F2 的最后一个簇的簇号还需要读一级索引表。综上，需要的时间不相同。

26. **15.** 【解答】

1）在该文件系统中，目录项由文件名和索引节点号构成。由图(a)可知，stu 目录下有两个文件，分别是 course 和 doc。由图(b)可知，这两个文件分别对应索引节点号 2 和 10。因此，目录文件 stu 中两个目录项的内容是

文 件 名	索引节点号
course	2
doc	10

2）由图(b)可知，文件 doc 和文件 course1 对应的索引节点号都是 10，说明 doc 和 course1 两个目录项共享同一个索引节点，本质上对应同一个文件。而文件 course1 存储在 30 号磁盘块，因此文件 doc 占用的磁盘块的块号 x 为 30。

3）需要读 2 个磁盘块。先读 course1 的索引节点所在的磁盘块，再读 course1 的内容所在的磁盘块。目录文件 course 的内容已在内存中，即 course1、course2 对应的目录项已在内存中，根据 course1 对应的目录项可以知道其索引节点号，即可读入 course1 的索引节点所在的磁盘块；根据 course1 的索引节点可知该文件存储在 30 号磁盘块，因此可再读入 course1 的内容所在的磁盘块。

4）存取 course2 需要使用索引节点的一级和二级间接地址项。6MB 大小的文件需要占用 6MB/4KB = 1536 个磁盘块。直接地址项可以记录 10 个磁盘块号，一级间接地址块可以记录 4KB/4B = 1024 个磁盘块号，二级间接地址块可以记录 1024×1024 个磁盘块号，而 10 + 1024 < 1536 < 10 + 1024 + 1024×1024。因此，6MB 大小的文件，需要使用一级间接地址项和二级间接地址项（拓展：若文件的总大小超出 10 + 1024 + 1024×1024 块，则还需要使用三级间接地址项）。

27. 【考点：文件的操作】关于文件的操作、文件元数据（inode）以及系统级内存布局的微观机制，考虑如下并发场景：进程 P_1 通过原始路径 `/dir/f1` 执行 `open` 操作获得文件描述符 fd_1；随后进程 P_2 对该文件创建了软链接（Symbolic Link）`/dir/f2`，并通过此软链接执行 `open` 操作获得 fd_2。最后，P_1 执行 `unlink("/dir/f1")` 删除了原文件目录项。下列关于底层机制及系统状态的叙述中，正确的是（ ）

A. 由于 P_1 和 P_2 最终打开的是同一个目标文件，因此系统会在“系统级打开文件表（Open File Table）”中为它们分配同一个共享表项，两者共享文件读写偏移量

B. P_2 通过软链接打开文件时，虚拟文件系统（VFS）会在内存中为“软链接文件本身”和“目标数据文件”分别创建两个独立且不同的 vnode/inode 缓存结构

C. 执行 `unlink` 后，由于目标文件的硬链接数（i_nlink）变为 0，操作系统将立即回收该文件在外存中的数据块，从而导致 P_1 随后的 `read` 操作直接触发异常

D. 执行 `unlink` 后，目标路径 `/dir/f1` 被销毁，`/dir/f2` 成为“死链接（悬空链接）”，因此 P_2 通过 fd_2 发起的后续 `read` 操作必然失败报错

【答案】： B

【解析】：

A 项错误：无论是通过原路径还是软链接，只要是两次独立的 `open` 操作，系统都会在“系统级打开文件表”中创建两个独立的表项，两者拥有各自独立的读写指针（Offset）。只有通过 `fork` 或 `dup` 机制才会共享同一个系统级打开文件表项

B 项正确：软链接本质上是一个独立的物理文件，其数据块中存储的是目标文件的路径字符串。因此，它拥有自己独立的 inode。当 VFS 解析软链接时，会在内存目录项缓存（dcache）和 inode 缓存（icache）中同时实例化软链接的 vnode 以及被指向目标的 vnode

C 项错误：这是文件删除机制的经典考点。文件的真正物理删除需要满足两个条件：硬链接数（`i_nlink`）等于 0，且内存中打开该文件的引用计数（`i_count`）等于 0。虽然 `unlink` 让 `i_nlink` 归零，但由于 P_1 和 P_2 仍打开着该文件（`i_count > 0`），磁盘数据块不会被立即回收，P_1 依然可以正常读写

D 项错误：P_2 在执行 `open` 阶段时，路径解析已经完成，其获得的文件描述符 fd_2 在内核中已经

直接指向了目标文件的 inode。即使原路径被 `unlink` 导致软链接变为死链接，这只会影响后续试图打开该软链接的新进程。已经成功打开该文件的 P_2 其底层 fd 绑定的是 inode 而不是路径，因此 `read` 依然正常执行

28. 【考点：虚拟文件系统】虚拟文件系统（VFS）与文件系统挂载（Mounting）技术使得 Linux 等操作系统能够屏蔽底层存储介质的差异。关于文件系统全局结构与虚拟化抽象的深度机制，下列说法中错误的是（ ）

    A. 虚拟文件系统（VFS）采用面向对象的思想，即使底层的 FAT32 文件系统在外存中并没有真正的物理 inode 结构，VFS 也会在文件被访问时于内存中为其临时合成出对应的 vnode/inode 结构
    B. 当管理员将一个 Ext4 格式的 U 盘挂载（Mount）到系统已有目录 `/mnt/data` 上时，该目录下原有的文件与子目录在挂载期间将被系统隐藏而无法被直接访问，直到 U 盘被卸载（Unmount）
    C. 若进程执行跨越不同文件系统挂载点的数据移动（如 `mv /home/a.txt /mnt/data/`），系统无法仅仅通过修改目录项来完成转移，必须在底层执行完整的新建文件、拷贝数据以及删除原文件的微观操作序列
    D. 为了加速文件路径的解析过程，VFS 在内存中维护了目录项缓存（dentry cache），为了防止断电导致目录数据丢失，系统后台守护进程会定时将内存中的 dentry cache 全量写回外存的特定隐藏区域中

    【答案】： D
    【解析】：
    A 项正确：VFS 的核心是提供统一的模型（如 inode、dentry、superblock）。对于不支持物理 inode 的外存系统（如基于 FAT 表的系统），VFS 驱动层会在读取目录项时，动态在内存中“伪造”出符合 VFS 标准的 vnode/inode 结构，这是 VFS 多态性的体现
    B 项正确：挂载操作的作用机制是利用新文件系统的根目录结构“遮盖（Overmount）”掉挂载点目录。挂载点原有的目录项在 VFS 的挂载表中被压入栈底，用户空间的路径解析会被重定向到新的文件系统中，原文件依然安全存放在原外存块中，只是暂不可见
    C 项正确：在同一个文件系统（同一分区）内移动文件，只需在原目录删除目录项，在新目录增加目录项即可（仅修改元数据）。但跨越文件系统时，由于 inode 编号仅在单个文件系统内部有效，无法跨区硬链接，必须发生物理数据的拷贝
    D 项错误：dentry cache（目录项缓存）纯粹是为了加速内存中路径解析而设立的软件数据结构（将字符串路径映射为 inode 内存指针）。它仅仅存在于内存中，在外存物理结构中并没有对应的数据存储区。外存中的目录是以普通文件数据块的形式存放（目录文件）。因此不存在将 dentry cache 全量写回外存的做法，修改的目录信息只需写回原目录文件的数据块即可

29. 【考点：文件的物理结构】关于文件的物理结构及外存空闲空间管理的微观同步操作。某文件系统采用 UNIX 风格的混合索引结构（FCB 包含 10 个直接、1 个一级间接、1 个二级间接、1 个三级间接指针），磁盘逻辑块大小为 4KB，地址指针大小为 4B。外存空闲空间采用位示图（Bitmap）管理，且位示图核心部分已载入内存。当前某普通文件 F 的大小恰好为 40KB，且其对应的 inode 已缓存在内存中。现某进程以同步阻塞 I/O 方式向文件 F 的末尾追加写入了 10 字节的数据。为了保证在系统突然断电时文件系统全局结构的完整性与一致性（假设不考虑操作系统的缓冲延迟写机制，所有变更均需立刻落盘），本次追加操作最少需要触发多少次外存块的“写回（Write）”动作？（不考虑父目录最后修改时间的更新）

    A. 2 次
    B. 3 次
    C. 4 次
    D. 5 次

    【答案】： C
    【解析】：
    计算当前文件状态：盘块大小 4 KB，文件目前 40 KB。40 KB ÷ 4 KB = 10 块。这意味着文件的 10 个直接指针恰好全部被使用完毕
    分析追加写需求：再追加写 10 字节，必须分配第 11 个逻辑块。由于直接索引已满，必须启用一级间接指针
    确定需要新分配的物理块数量：不仅需要分配 1 个用来存放这 10 字节的数据盘块，还需要分配 1

个用来存放盘块地址的索引盘块（即一级间接块本身）
推演微观写回序列（需同步落盘的结构）：
动作一：在内存位示图中找到 2 个空闲块并标记为已分配。对应的 Bitmap 位示图块 必须写回外存（第 1 次外存写操作）
动作二：将 10 字节数据写入新分配的数据盘块中，并将该数据块写回外存（第 2 次外存写操作）
动作三：将数据盘块的地址填入新分配的一级间接索引块中，并将该索引块写回外存（第 3 次外存写操作）
动作四：文件在内存中的 inode 被修改（文件大小增加了 10 字节，且启用了一级间接指针指向新索引块），更新后的 inode 元数据块必须写回外存（第 4 次外存写操作）
综上所述，在此边界条件下，为了分配并写入极小的一段数据，系统在底层至少需要执行 4 次 外存写操作。正确答案为 C

30. 【考点：树形目录】硬链接与软链接在树形目录的构建与文件保护机制中扮演着核心角色。关于它们对文件元数据（inode）、目录操作及逻辑拓扑影响的深度分析，下列说法中错误的是（ ）

A. 在同一文件系统中为一个目标文件建立硬链接时，操作系统的核心动作是在指定目录的数据块中增加一个“文件名 → 相同的 inode 编号”映射记录，并不占用新的存储数据盘块
B. 如果用户试图规避文件保护权限，对一个只有“读权限”的目标文件创建软链接，由于软链接自身默认具备最高权限（如 777 权限），该用户可以通过该软链接成功越权修改目标文件的内容
C. 主流操作系统严格禁止普通用户为“目录（Directory）”创建系统级硬链接，这一强制约束的根本原因是为了防止在树形目录系统及其全局遍历（如文件检索、磁盘空间检查）中出现灾难性的循环路径和死锁
D. 假设存在软链接文件 `S_Link` 指向普通文件 `Target.txt`，若进程通过 `S_Link` 写入了大量新数据，导致最后修改时间发生变更，此时只有目标文件 `Target.txt` 的 inode 时间戳会更新，软链接文件 `S_Link` 自身的最后修改时间（mtime）不会改变

【答案】： B
【解析】：
A 项正确：硬链接的本质是多个目录项（dentry）指向同一个 inode 号。建立硬链接仅在目录文件中追加了一条记录，并未创建新的文件控制块（FCB/inode）和数据块，因此不额外消耗存储数据的空间
B 项错误：这是一个经典的越权陷阱题。当进程通过软链接访问文件时，操作系统的 VFS 在路径解析的最后阶段，最终都会回溯到目标物理文件的 inode，并基于目标 inode 所记录的权限信息以及当前用户的凭证进行权限检查。即使软链接文件自身显示拥有 `rwxrwxrwx` 权限，也绝对无法突破目标文件自身的权限保护壁垒
C 项正确：树形目录（严格说是层次化的有向无环图 DAG）如果允许目录拥有硬链接，极易打破无环结构。如果产生父子目录循环（如目录 A 的硬链接放入了其子目录 B 中），会导致系统工具（如 `fsck` 修复工具、`find` 遍历等）陷入无限死循环。因此除系统本身自动维护的 `.` 和 `..` 外，严禁对目录创建硬链接
D 项正确：软链接的数据内容仅仅是“目标文件的路径字符串”。通过软链接修改数据，数据是穿透写入到了目标数据块中。由于软链接自身的“路径字符串”内容并没有被改变，因此软链接文件自身 inode 中的最后修改时间（mtime）保持不变，只有目标文件的 inode mtime 发生更新